

Einführung

DHBW Stuttgart, 2024

Christian Holz

christian.holz@lehre.dhbw-stuttgart.de

Inhalt

- Definition eines Betriebssystems
- Aufgaben und Anforderungen
- Abstraktion
- Historischer Rückblick
- Arten von Betriebssystemen
- Grundlegende Konzepte
- Architekturen von Betriebssystemen

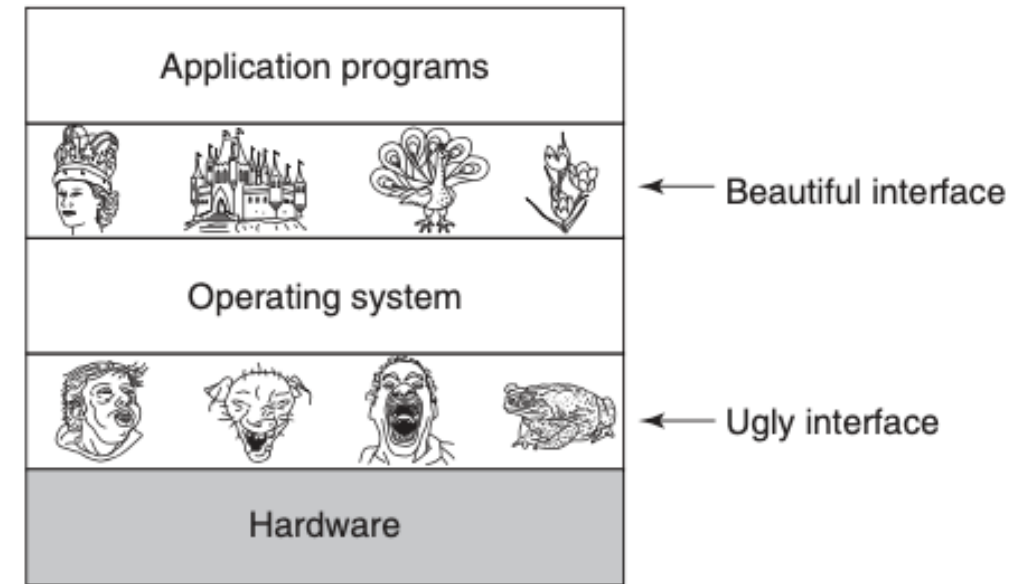
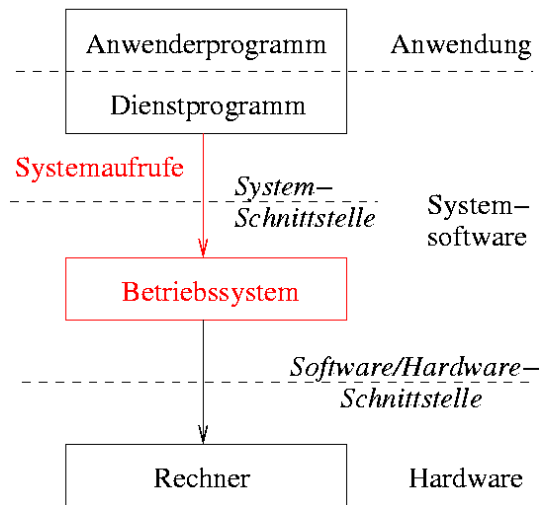
Definition eines Betriebssystems

- Was ist ein Betriebssystem?
- Einfache Definition (DIN 44300):
„Als Betriebssystem bezeichnet man die Programme eines digitalen Rechensystems, die zusammen mit den Eigenschaften dieser Rechenanlage die Basis der möglichen Betriebsarten des digitalen Rechensystems bilden und die insbesondere die Abwicklung von Programmen steuern und überwachen.“
- Genauere Definition:
Ein Betriebssystem ist die Gesamtheit der anwendungsunabhängigen Programmteile, die die Benutzung von Betriebsmitteln steuern und die zugehörigen Hardware-Ressourcen verwalten.
- Welche Betriebsmittel liegen in einem heutigen Rechner vor?
- Zu einem Betriebssystem wird gelegentlich auch die Bereitstellung von Dienstprogrammen (z.B. Editor, Benutzershell, Grafische Oberfläche, Compiler, Linker, ...) gezählt

Definition eines Betriebssystems

- Die Schnittstelle zwischen Anwendung und Betriebssystem heißt **Systemschnittstelle** und stellt für den Anwender eine abstrakte Maschine dar

einfaches Schichtenmodell:



A.S. Tanenbaum – Modern Operating Systems (2014)

Aufgaben und Anforderungen

Anforderungen an ein Betriebssystem

- **Hohe Zuverlässigkeit**

- Sicherheit
- Verfügbarkeit
- Robustheit
- Schutz von Benutzerdaten

- **Hohe Benutzerfreundlichkeit**

- Angepasste Funktionalität
- Einfache Benutzerschnittstelle
- Hilfestellungen

- **Hohe Leistung**

- Gute Auslastung der Ressourcen des Systems
- Kleiner Verwaltungsoverhead
- Hoher Durchsatz
- Kurze Reaktionszeit

- **Einfache Wartbarkeit**

- Einfache Upgrades
- Einfache Erweiterbarkeit
- Portierbarkeit

Aufgaben eines Betriebssystems

- Benutzerschnittstelle
 - Kommandosprache und Kommandointerpreter – ggf. graphische Oberfläche
 - Programmierschnittstelle zur abstrakten Nutzung von Betriebsmitteln
 - Bereitstellung eines Dateisystems
- Verwaltung der Betriebsmittel (Systemressourcen)
 - Speicherverwaltung (Swapping, Paging, virtueller Speicher)
 - Prozessverwaltung (Scheduling)
 - ggf. auch Prozessorverwaltung (bei Mehrprozessorbetrieb)
 - Geräteverwaltung
- Welche weiteren Aufgaben hat ein Betriebssystem?

Aufgabe: Beispiele von Betriebssystemen

- Welche Betriebssysteme kennen Sie?
- Worin unterscheiden sie sich?
- Versuchen Sie eine einfache Klassifikation zu erstellen!

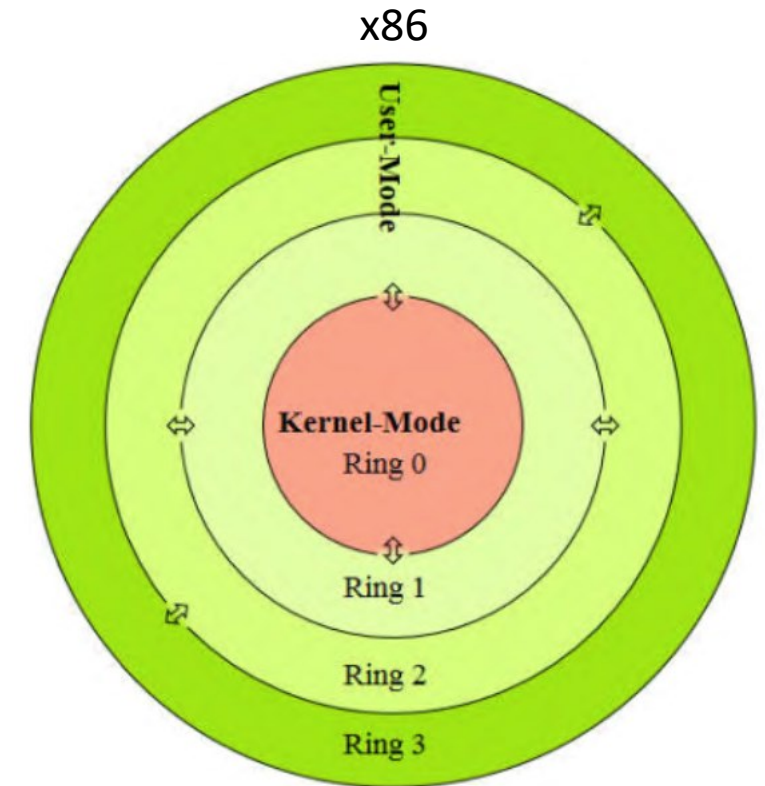
Abstraktion

Ziele der Abstraktion

- Ein Betriebssystem stellt dem Nutzer eine **erweiterte abstrakte Maschine** bereit
- Ziele:
 - Abstraktion von detailliertem Verhalten des zugrunde liegenden Rechners auf möglichst hohem Niveau
 - Verstecken realer Hardware-Eigenschaften vor dem Benutzer
- Beispiel: Manipulation des Dateisystems
 - Beim Anlegen einer neuen Datei soll der Benutzer sich nicht um die physikalischen Details der Hardware (Festplatte und Controller) kümmern
 - Betriebssystem übernimmt Zerteilung der Datei in Blöcke, Auswahl von Spur und Sektor für jeden Block, ...

Abstraktion der CPU

- Aus Hardwaresicht erfolgt die Ausführung von Maschinenbefehlen auf verschiedenen Sicherheitsstufen, die als **Ringe** bezeichnet werden
 - Ring 0 ist der Kernel-Modus (supervisor mode) und enthält privilegierte Befehle (z.B. zur Interruptverwaltung)
 - Ringe 1 und 2 sind für Gerätetreiber vorgesehen
 - Ring 3 enthält nicht-privilegierte Befehle für das Anwendungsprogramm
- Moderne Betriebssysteme verwenden oft nur Ringe 0 (Betriebssystem) und 3 (Benutzeranwendungen)



Betriebssystem API

- Dem Programmierer stellt das Betriebssystem eine **abstrakte Programmierschnittstelle** (API = Applications Programmer Interface) zur Verfügung
- Hierzu bietet es einen Satz von Kommandos (**Systemaufrufe**) an, über die geräteunabhängig z.B. auf von einem Programm auf eine Datei oder ein Ein-/Ausgabegerät zugegriffen werden kann
- Einige Systemaufrufe in Unix/Linux (längere Liste [hier](#)):
 - Dateizugriff: `open`, `close`, `read`, `write`, `stat`, `lseek`
 - Verzeichniszugriff: `opendir`, `closedir`, `readdir`
 - Prozessverwaltung: `fork`, `exec`, `wait`, `exit`, `getpid`, `getppid`
 - Zugriff auf Systeminformationen: `gettimeofday`, `localtime`

Beispiel

- Lesen einer Datei in Unix

```
fd = open("/tmp/log.txt", O_RDONLY);  
count = read(fd, buffer, nbytes);
```

- Was leistet der Aufruf von read()?
- Wie könnte er intern ablaufen?

Historischer Rückblick

Die Geschichte des Computers Teil I

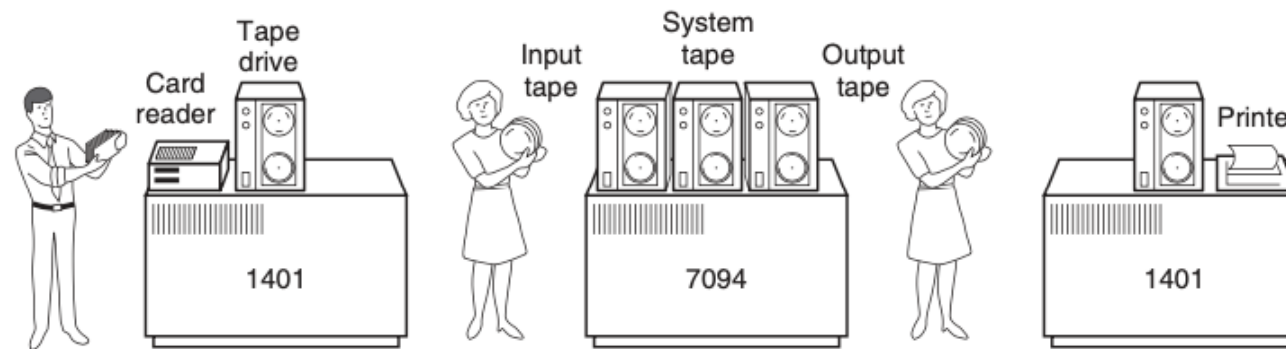
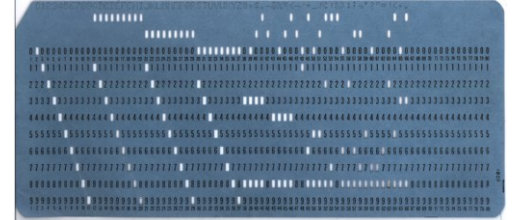
<https://www.youtube.com/watch?v=8wIRT0U9QiM>

Überblick

- Früher: Hardwarespezifische Betriebssysteme (z.B. VAX VMS, IBM OS/360, MS-DOS)
- Heute: überwiegend hardwareunabhängige, z.B. UNIX-Derivate wie AIX, Solaris, Linux
Ausnahme: Windows (bislang überwiegend nur für x86 bzw. x86-64 CPUs)
- Verschiedene Entwicklungsstadien der Betriebssysteme
 - Einfache Stapelverarbeitungssysteme
 - Mehrprogrammfähige Stapelverarbeitungssysteme
 - Timesharing-Systeme
 - Systeme mit graphischen Benutzeroberflächen
 - Netzwerkbetriebssysteme
 - Mobile Computer
 - Mehrkern-Betriebssysteme

Transistoren & Stapelverarbeitungssysteme (1955-1965)

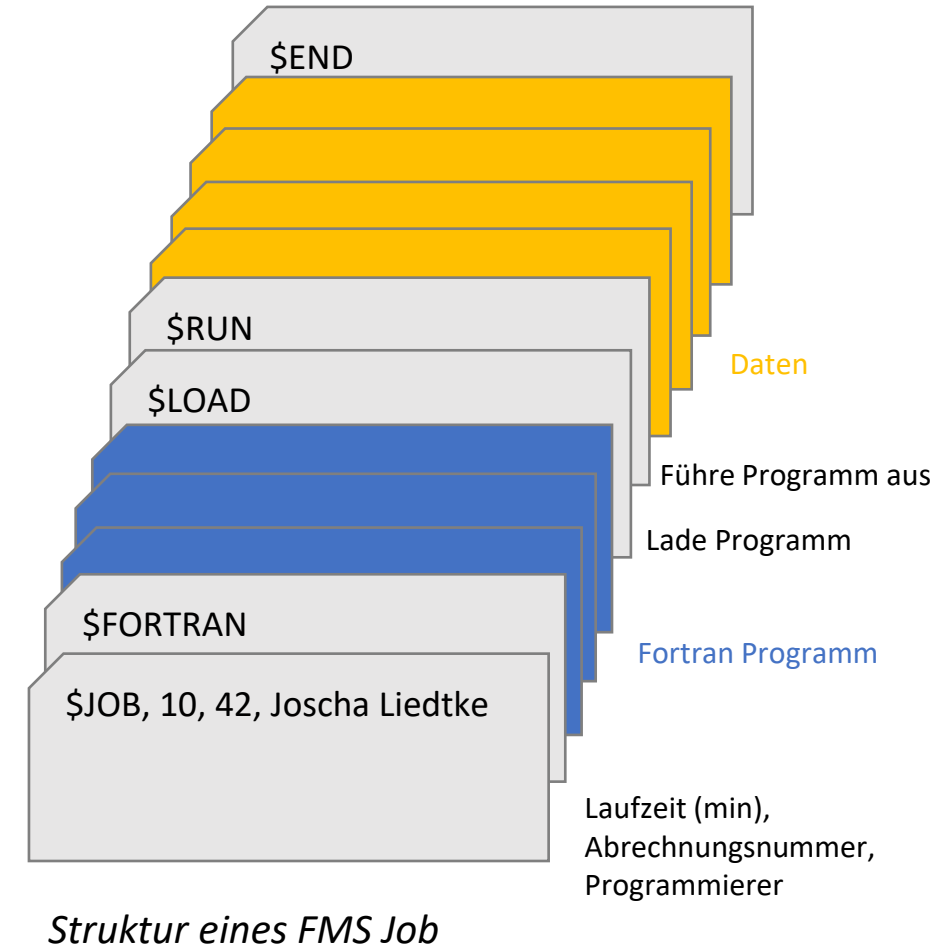
- Erfindung des Transistors ermöglichte zuverlässige Computer
- Erstellung von Programmen (Jobs) auf Lochkarten
- Optimierung durch separate Rechner für Ein-/Ausgabe
 - Eingaberechner für das Einlesen der Lochkarten, Programm und Daten werden auf Magnetband geschrieben
 - Zentraleinheit für Ausführung der Jobs, Minimalbetriebssystem (Monitor) liest und startet Jobs sequentiell vom Band, Ergebnisse wieder auf Band
 - Ausgaberechner, der Ergebnisse von Band zum Drucken aufbereitet



A.S. Tanenbaum – Modern Operating Systems (2014)

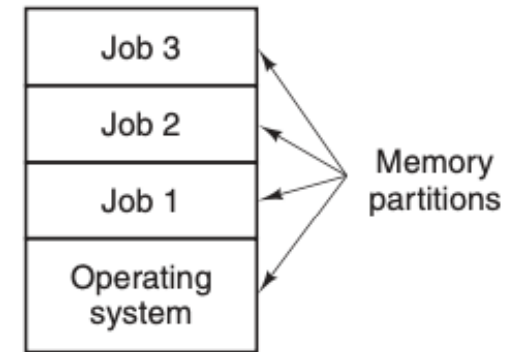
Transistoren & Stapelverarbeitungssysteme (1955-1965)

- Abarbeitungsmethodik eines Jobs ist der Vorgänger eines heutigen Kommandointerpreters (Shells)
- Rechner dieser Generation (Mainframes) wurden z.B. in Forschungseinrichtungen, etwa zur Lösung großer Differentialgleichungen eingesetzt
- Programmiert wurde in FORTRAN, COBOL oder Assembler
- Typische Betriebssysteme waren FMS (FORTRAN Monitor System) und IBSYS, das IBM Betriebssystem des Großrechners 7094



Integrated Circuits & Multiprogrammierung (1965-1980)

- Durch bessere Hardware blieb Rechenzeit ungenutzt aufgrund des Wartens der CPU auf Beendigung von I/O Operationen → Nutzen der Wartezeit für andere Jobs
- Welche Programme sind typischerweise (nicht) I/O-lastig?
- Jobs werden aus Effizienzgründen nicht sequentiell ausgeführt, stattdessen werden quasi-parallel (Multiprogramming) verzahnt abgearbeitet (Wechsel beim Warten auf I/O)
- Eigene Speicherpartition für jeden aktiven Job
- Bekanntes Beispiel: IBM OS/360 (1967)
- **Spooling** (Simultaneous Peripheral Operation On Line) erlaubt das automatische nachladen von Jobs in Speicherpartitionen
- Vor-/Nachteile? Was geschieht beim Jobwechsel?



A.S. Tanenbaum – Modern Operating Systems (2014)

Integrated Circuits & Multiprogrammierung (1965-1980)

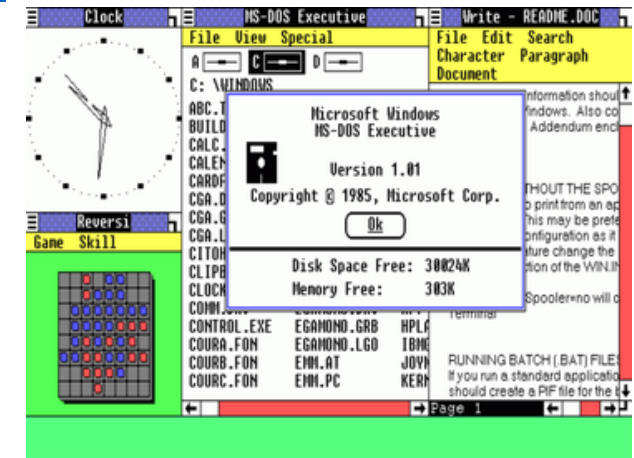
- Nachteil bisheriger Betriebssysteme: Kein interaktives Arbeiten/Debuggen mehrerer Benutzer möglich
- Idee: Interaktives Arbeiten eines Benutzers erfordert nicht die komplette Rechenzeit eines Rechners
- Jeder Benutzer ist online mit dem Rechner (**Timesharing-System**) über ein eigenes Terminal verbunden (auch als Dialogsystem bezeichnet)
- Bei schnellem Umschalten bemerkt der Einzelnutzer nicht, dass er die Maschine nicht für sich allein hat
- Bekanntes Beispiel: MULTICS (MULTiplexed Information and Computing Service, 1969)
- Einzelnutzer Version von MULTICS: UNIX (1971)

Personal Computer (seit 1980)

- GUI (*Graphical User Interface*): Fenster, Icons, Menüs, Maus
- zuerst eingeführt beim Apple Macintosh (1984)
- später Microsoft Windows (seit 1985): Graphische Umgebung mit Version 3.1 zunächst aufsetzend auf MS-DOS
- seit Windows NT: Betriebssystem und GUI stark miteinander verschränkt
- Unix/LINUX
 - GUI als Aufsatz auf das Betriebssystem
 - X-Windows-System/Wayland: Grundlegende Funktionen zur Fensterverwaltung
 - Komplette GUI-Umgebungen basierend auf X-Windows: z.B. KDE, GNOME



Erste Macintosh, Quelle: [Wikipedia](#), [CC BY-SA 3.0](#)



Windows 1.0. Quelle: [Wikipedia](#)

Netzwerkbetriebssysteme (seit 1985)

- Mehrere vernetzte Rechner im lokalen Netzwerk
- Anmelden auf entfernten Rechnern vom lokalen Rechner aus möglich (über Dienstprogramme wie telnet, rsh, ssh, ...)
- Datenaustausch durch Verwendung von Standardprotokollen (wie z.B. ftp, scp, ...) möglich
- Normales Betriebssystem auf jedem Einzelrechner
→ neu ist auf unterster Ebene lediglich eine zusätzliche Betriebssystemschicht, die über einen Netzwerkcontroller auf Netzressourcen zugreift.

Mobile Computer (seit 1990er)

- Erstes Smartphone: Nokia N9000 (1996) mit DOS basiertem Betriebssystem PEN/GEOS
- Dominierendes Smartphone Betriebssystem bis 2010: Symbian OS (EPOC basiert)
- Erstes iPhone (2007) mit Unix/BSD basierten Betriebssystem iOS
- Linux basiertes Betriebssystem Android (2008)



Nokia N9000, Quelle: [Wikipedia](#), [CC BY 2.0](#)



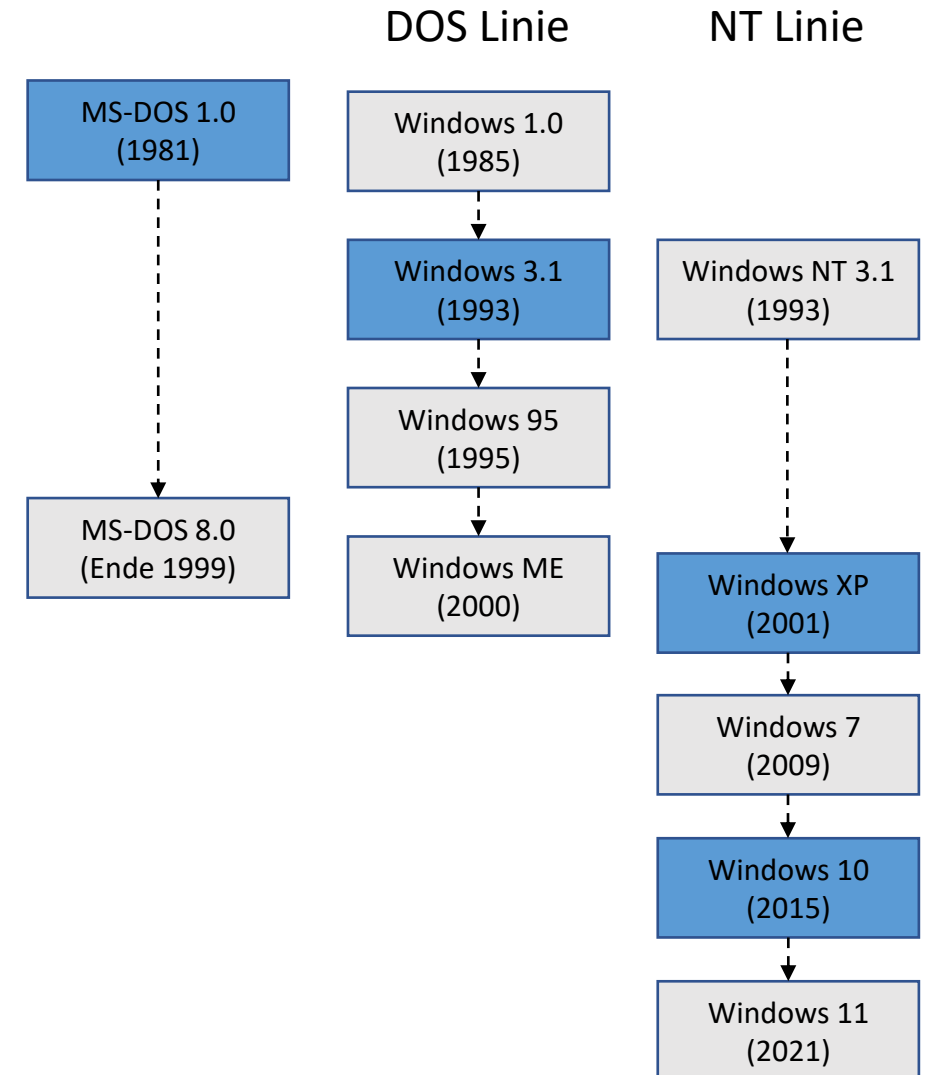
Erste Generation des iPhone

Aktuelle Betriebssysteme

- Betriebssysteme für **Mehrkern-Prozessoren**
- Aufteilung der Prozesse/Threads auf vorhandene Kerne (typ.: 4 bis 8)
- Jeder Kern stellt eigene Recheneinheit dar, Zugriff auf gemeinsame Ressourcen muss verwaltet werden
- Theoretisch *n-fache Rechenleistung bei n Kernen* (abhängig von Parallelisierung der Software)
- mehrere Kerne unterstützen z.B. Linux (mit SMP Kernel) und Windows (ab XP)
- Virtueller Speicher

Geschichte von Windows

- MS-DOS 1.0
 - basiert auf QDOS (Quick and dirty Operating System)
 - Wichtigste Verbesserung: FAT (File Allocation Table)
- Windows 3.1
 - Durchbruch im Bereich der grafischen Betriebssysteme
 - Stabile API; ähnlich in Windows 3.x/9.x und NT
- Windows XP
 - Vollständiger Umstieg auf Windows NT (New Technology)
- Windows 10
 - Rudimentäre Unterstützung von ARM CPUs, vorher überwiegend x86
 - Experimentelle Unterstützung von WSL (Windows-Subsystem for Linux)



Geschichte von UNIX

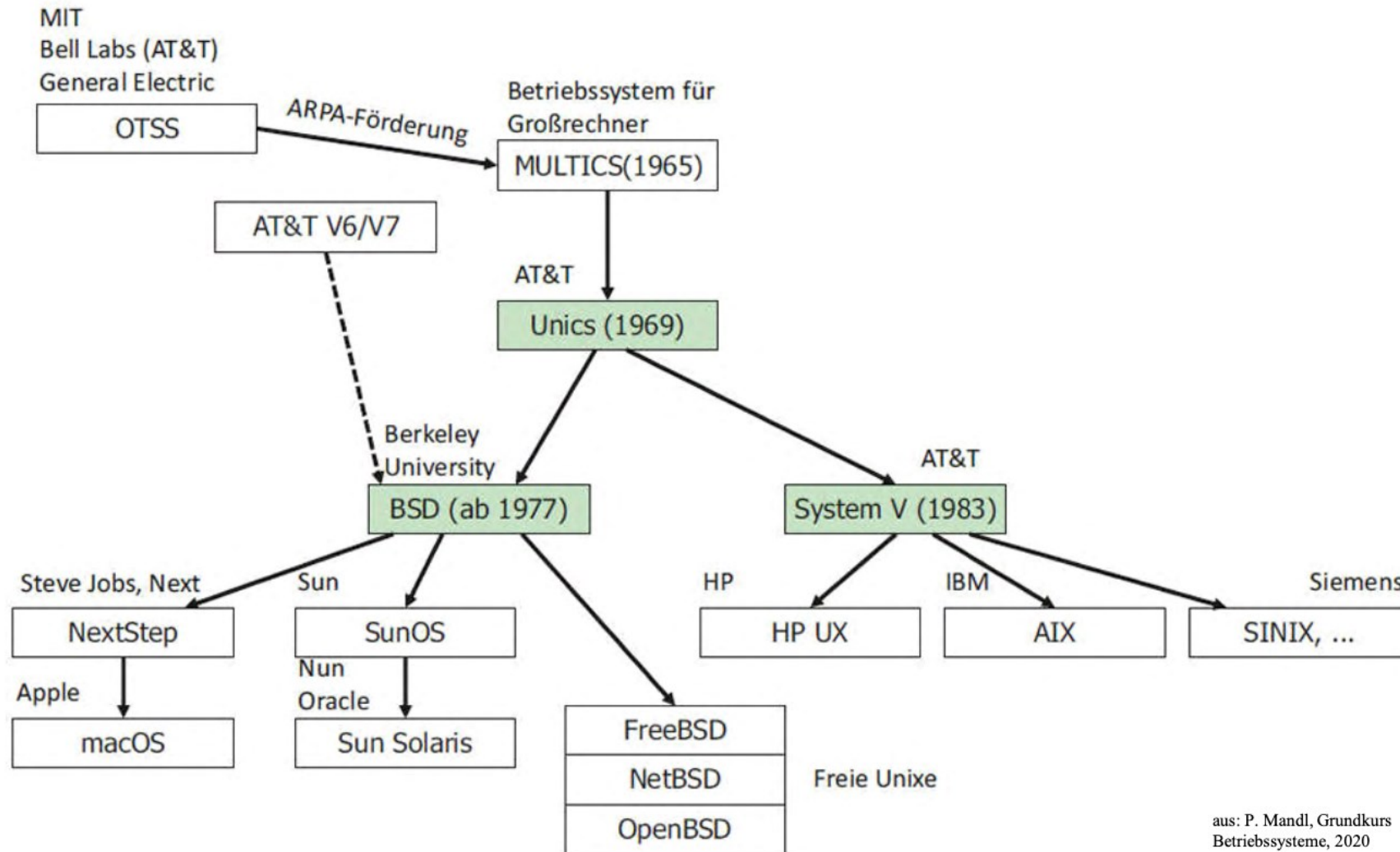
- 1969 von K. Thompson in den AT&T Bell Laboratories für PDP-7 entwickelt und in Maschinensprache implementiert
- 1973 von D. Ritchie für PDP-11 größtenteils in C umgeschrieben
 - leichte Portierbarkeit, nur kleiner maschinenabhängiger Betriebssystemkern
- 1977 - 1995 zwei UNIX Hauptlinien
 - UNIX System V (AT&T)
 - BSD4.x UNIX (Berkeley Software Distribution)
- 1993 FreeBSD resultiert aus dem BSD Projekt und ist Grundlage für OS X, iOS 7



K. Thompson und D. Ritchie

Die Entstehung von UNIX: <https://www.youtube.com/watch?v=OdJFi8fTsxg>

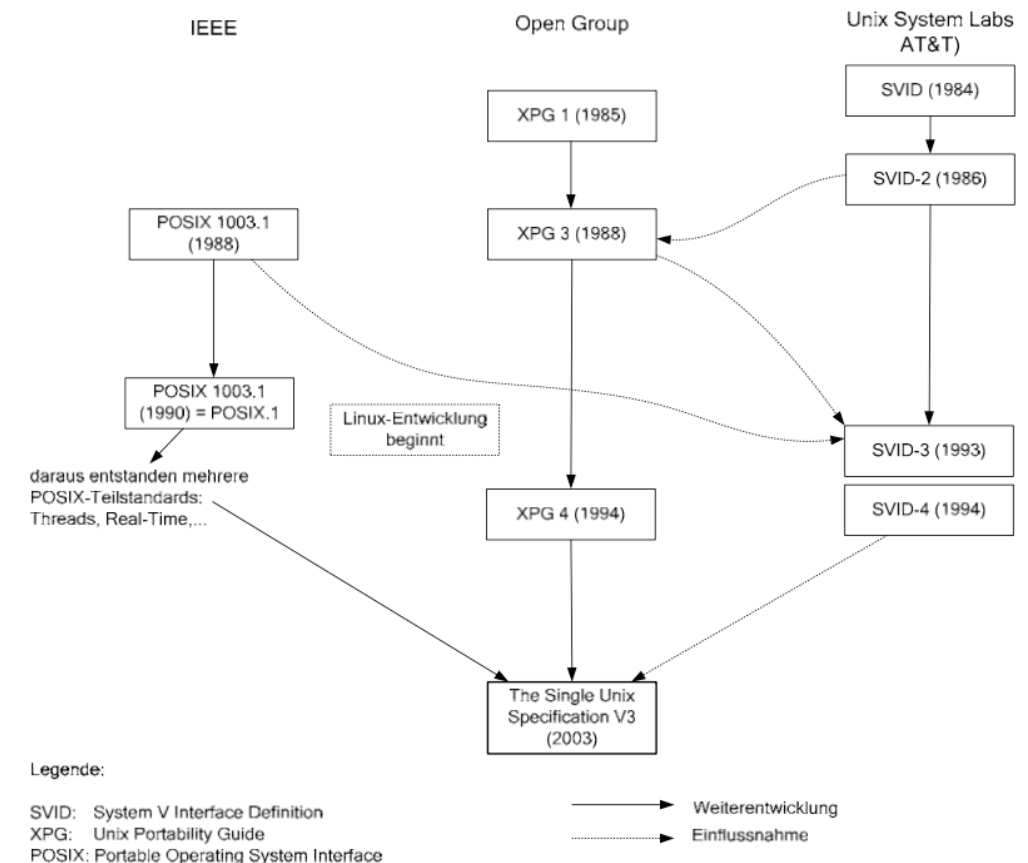
Geschichte von UNIX



aus: P. Mandl, Grundkurs
Betriebssysteme, 2020

Standardisierung von Unix

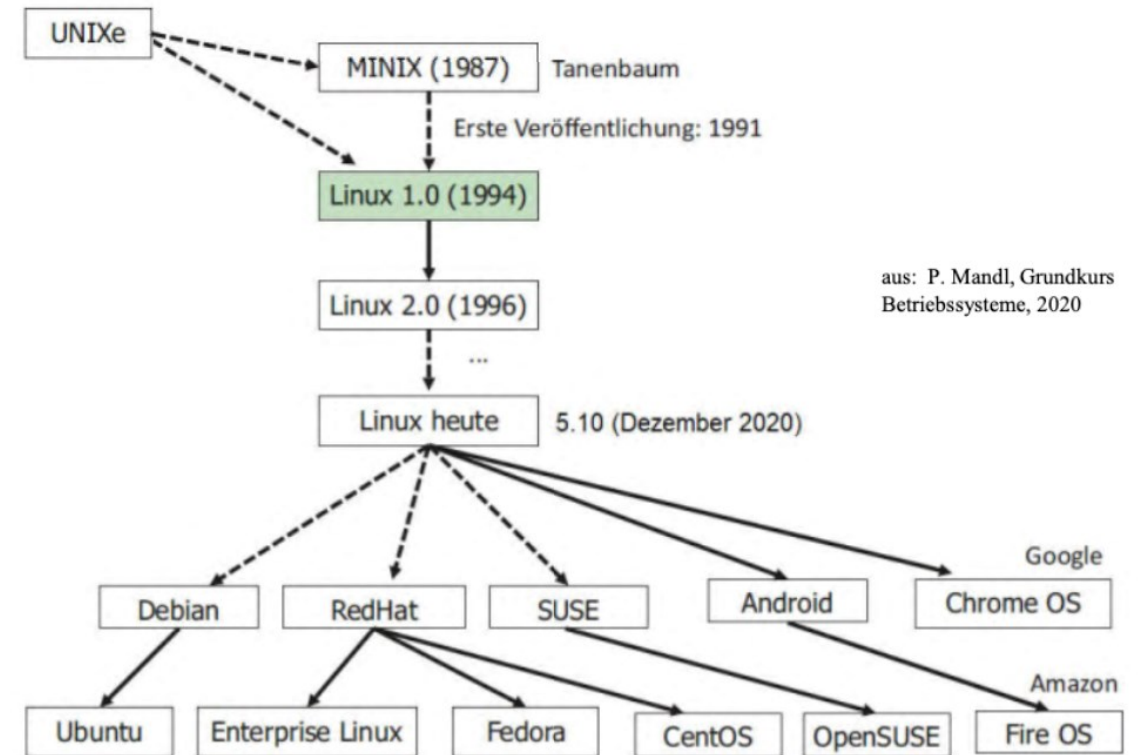
- 1988 erster UNIX-Standardisierungsvorschlag von IEEE: POSIX (Portable Operating System Interface)
- Seit 1995: Open Group legt UNIX Standards fest, u.a.: SUSv2, SUSv3, SUSv4 (Single Unix Specification)
- SUSv4 ist internationaler ISO/IEC Standard
- die meisten Unix-Derivate (Solaris, IBM AIX, HP-UX, ...) sind SUSv4-konform



Quelle: P. Mandl – Grundkurs Betriebssysteme

Geschichte von Linux

- 1991 schreibt Linus Torvalds einen Betriebssystemkern Linux
- Linux erfüllt nur POSIX 1003.1 und SVID-4, ist nicht SUSv4-konform
- Linux wird als UNIX ähnliches Betriebssystem bezeichnet
- Viele Distributionen die auf Linux aufbauen, wie z.B. Ubuntu und Android

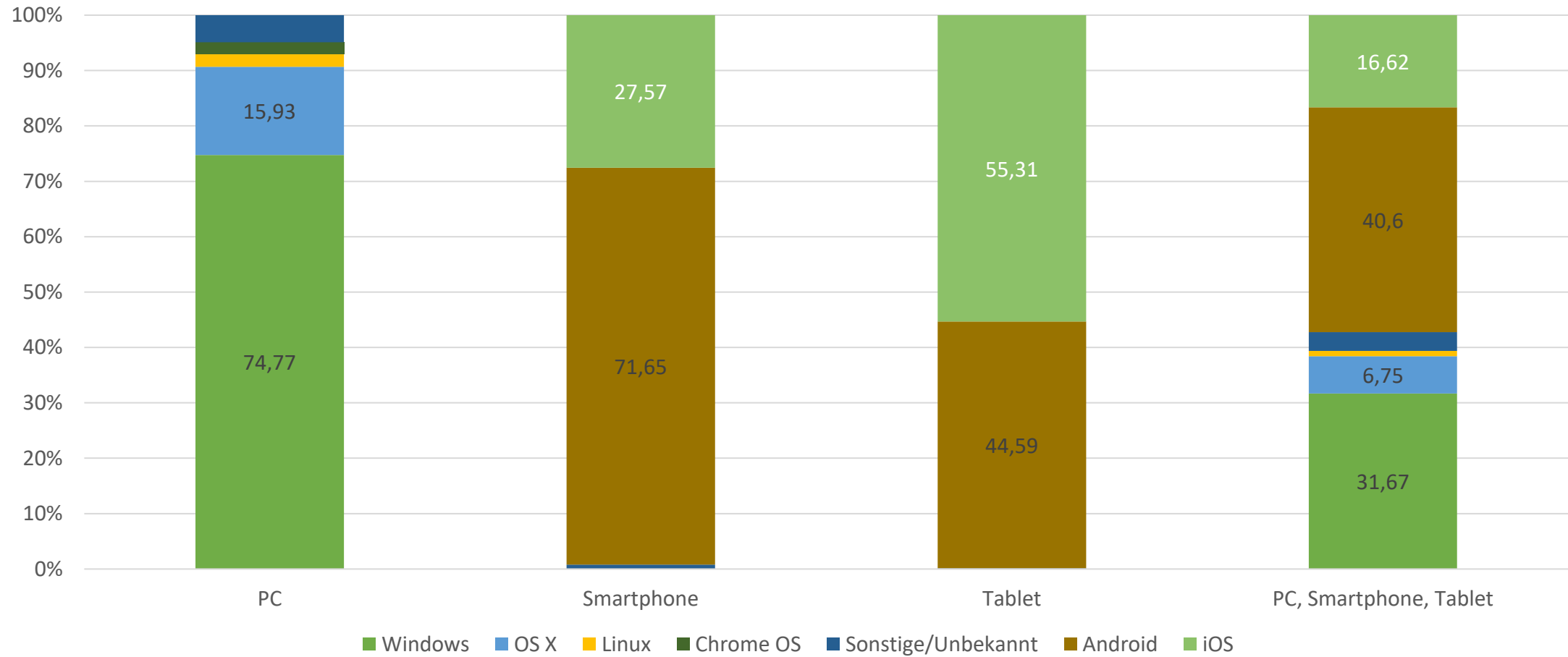


Quiz



"Dieses Foto" von Unbekannter Autor ist lizenziert gemäß [CC BY-SA-NC](#)

Markanteile der Betriebssysteme



Quelle: [StatCounter](#), Februar 2022, [CC BY-SA 3.0](#)

Arten von Betriebssystemen

Betriebssystem Zoo

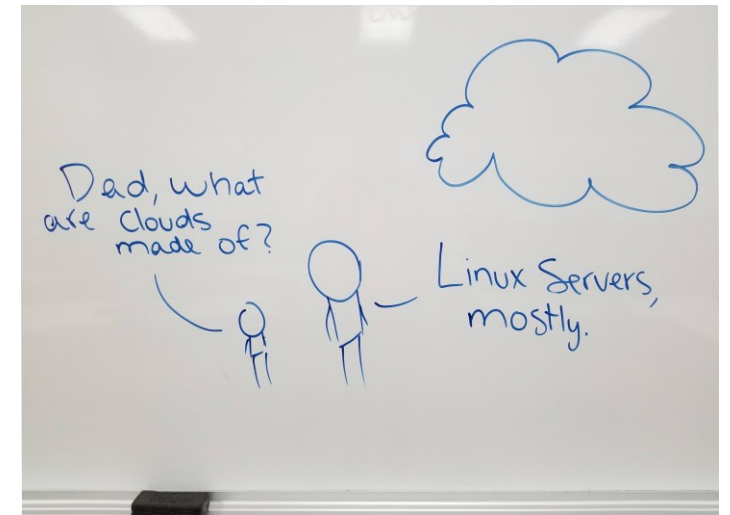
- In mehr als einem halben Jahrhundert haben sich eine Reihe von Betriebssystem Arten entwickelt
- Wichtige Betriebssystem Arten
 - Mainframe Betriebssystem
 - Server Betriebssystem
 - PC Betriebssystem
 - Betriebssysteme für Eingebettete Systeme
 - Echtzeit Betriebssysteme

Mainframe Betriebssysteme

- Einsatz: Webserver, E-Commerce, Business-to-Business
- Betriebssystem für Großrechner
- Viele Prozesse gleichzeitig mit hohem I/O Bedarf
- Drei Arten der Prozessverwaltung
 - Stapelverarbeitung (Batch Processing): Erledigung von Routineaufgaben ohne Benutzerinteraktion (Schadensübersichten, Verkaufsberichte)
 - Transaktionsverfahren/Dialogverarbeitung: Große Anzahl kleiner Aufgaben von vielen Nutzern (Überweisungen, Flugbuchungen)
 - Timesharing: Quasi-parallele Durchführung vieler Aufgaben durch mehrere Benutzer (Anfragen an Datenbank)
- Beispiel: IBM OS/390, z/OS
- Ablösung durch UNIX Varianten wie Linux

Server Betriebssystem

- Einsatz: Webserver, Printserver, Fileserver
- Betriebssysteme für sehr leistungsstarke PCs, Workstations oder auch Großrechner
- Typisches Ziel: Viele Benutzer gleichzeitig über Netzwerk bedienen
- Betriebssystem hat Aufgabe der Zuteilung von Hard- und Softwareressourcen
- Grenzen zum Mainframe Betriebssystem unscharf
- Beispiele: Unix, Windows Server



PC Betriebssystem

- Einsatz: Programmierung, Textverarbeitung, Spiele, Internetzugriff, ...
- Betriebssysteme für PCs und Laptops
- Meist nur 1 Benutzer erstes PC-Betriebssystem war CP/M (Control Program for Microcomputers, 1977), später verdrängt von MS-DOS (1982)
- Heute: mehrere Programme pro Benutzer gleichzeitig aktiv, ggf. Aufteilung der Prozesse auf vorhandene Kerne
- Zuteilung der Systemressourcen
- Beispiele: Linux, Windows, Mac OS X

Betriebssysteme für Eingebettete Systeme

- Eingebettete Systeme = Computer, die in Endgerät integriert sind und bei denen eine bestimmte Anwendung im Vordergrund steht (z.B. Fernseher, Mikrowelle, Aufzugsteuerung, ...)
- Meist Echtzeitanforderungen
- Kleiner Betriebssystemkern
- Wenig Ressourcen, kleiner Arbeitsspeicher (RAM), langsamer Prozessor
- Geringer Stromverbrauch gefordert
- Hohe Robustheit
- Schneller Systemstart
- Beispiele: Windows CE, iOS, Android, Windows 10 Mobile

Echtzeit Betriebssysteme

- Einsatz: Steuerung von Fertigungsanlagen, Medizintechnik, Luft- und Raumfahrt, komplexe Fahrerassistenzsysteme (ADAS)
- Anwendungen in denen Zeit ein kritischer Faktor ist
- Unterscheidung zwischen
 - Hartes Echtzeitsystem: Anwendung **muss** innerhalb eines Zeitfensters reagieren
 - Weiches Echtzeitsystem: Anwendung **sollten** innerhalb eines Zeitfensters reagieren
- Grenzen zu Eingebetteten Betriebssystemen unscharf (besonders zu weichem Echtzeitsystem)
- Beispiele: QNX, RTLinux, (Linux)

Grundlegende Konzepte

Prozess

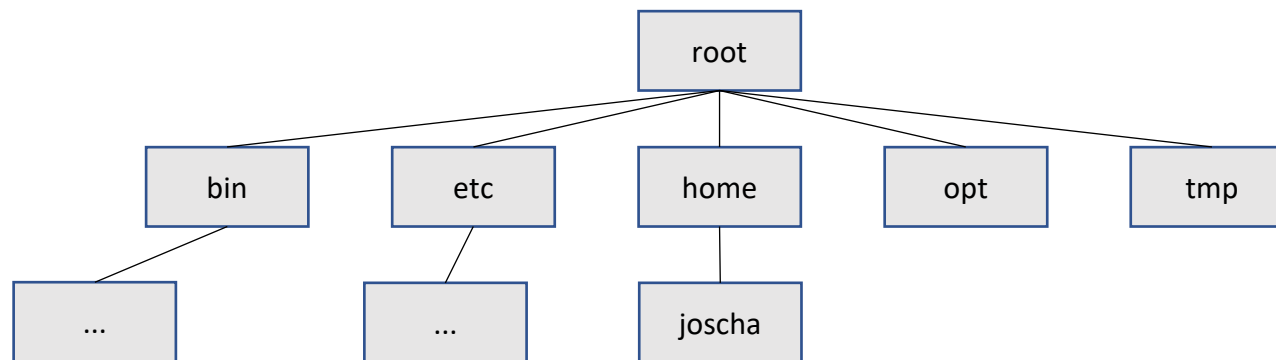
- Ein **Prozess** ist die Abstraktion eines ausgeführten Programmes
- Ein Prozess ist ein Behälter mit allen notwendigen Informationen zur Ausführung eines Programmes
- Jeder Prozess ist mit einem Speicherbereich assoziiert in dem er lesen/schreiben kann. Inhalte des Speicherbereichs
 - Programm Code
 - Programm Daten
 - Programm Stack
- Jeder Prozess ist mit Ressourcen assoziiert wie
 - Befehlszähler
 - Stackzeiger
 - Speicherinformationen
- Was können sich mehrere Prozesse des gleichen Programms teilen?

Speicherverwaltung

- Moderne Betriebssysteme erlauben die Ausführung von mehreren Programmen gleichzeitig
- D.h. der Speicherbereich von mehreren Prozessen befindet sich gleichzeitig im Arbeitsspeicher
 - Schutzmechanismus vor Wechselwirkung zwischen Prozessen notwendig
 - Mechanismus wird durch Hardware zur Verfügung gestellt, Betriebssystem kontrolliert
- Verwaltung des Speicherbereichs eines Prozesses durch **virtuellen Speicher**
 - Entkopplung von physikalischen Speicher und Prozessspeicher
 - Ermöglicht mehr Prozessspeicher als physikalisch vorhanden
 - Ermöglicht effizientere Speichernutzung
 - Isolation zwischen Prozessen

Dateisystem

- **Dateien** sind eine Folge von Bytes, die vom Betriebssystem auf einem Datenträger gespeichert werden
- Betriebssysteme stellen eine einfache Abstraktion zu den Hardwaredetails her
- Dateien werden organisiert, indem man **Verzeichnisse** anlegen kann. Ein Verzeichnis ist ein Katalog mit Inhalten, die aus Dateien und selbst wieder Verzeichnissen bestehen können

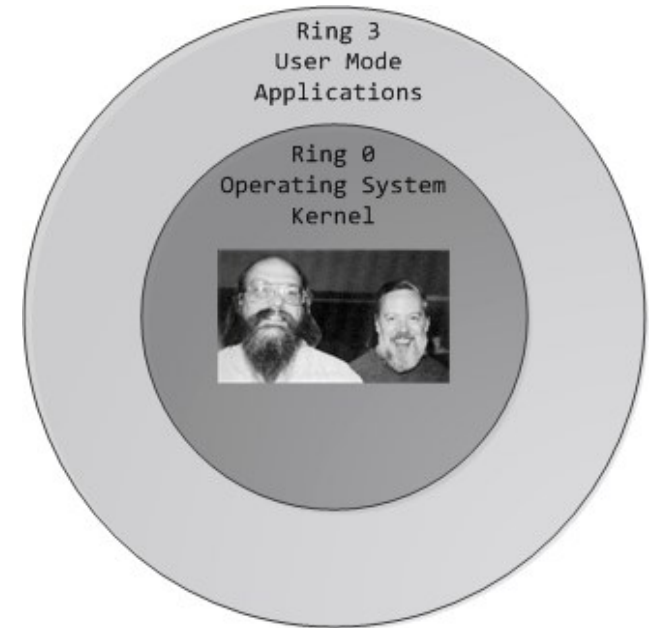


Benutzershell

- Je nach Definition nicht Teil des Betriebssystems (nutzt jedoch viele Features des Betriebssystems)
- Jedes Kommando, das der Benutzer in der Shell absendet, bewirkt, dass die Shell einen Kindprozess erzeugt, der das Kommando ausführt
- Die Shell wartet, bis der Kindprozess terminiert ist, dann wird wieder ein Prompt geschrieben
- Unterstützung von Skripten
- Beispiele:
 - Unix: sh, csh, tcsh, bash, zsh, ...
 - Windows: cmd.exe, Powershell (ab Windows 7)
- Was leistet das Unix Kommando `ls -l | grep pdf | sort` ?

Benutzer- und Kernelmodus

- Anwenderprogramme laufen in einem Betriebssystem im **Benutzermodus** (*user mode*) ab:
 - Zugriff nur auf eigene Anwendungsdaten
 - Zugriff nur auf zugewiesenen Teil des Arbeitsspeichers
 - Kein direkter Zugriff auf I/O-Geräte
 - Keine Ausführung privilegierter Maschinenbefehle
- Nur in den Routinen des Betriebssystems besteht Zugriff auf gemeinsame Ressourcen des Rechners, diese werden im **Kernelmodus** (*supervisor mode*) ausgeführt

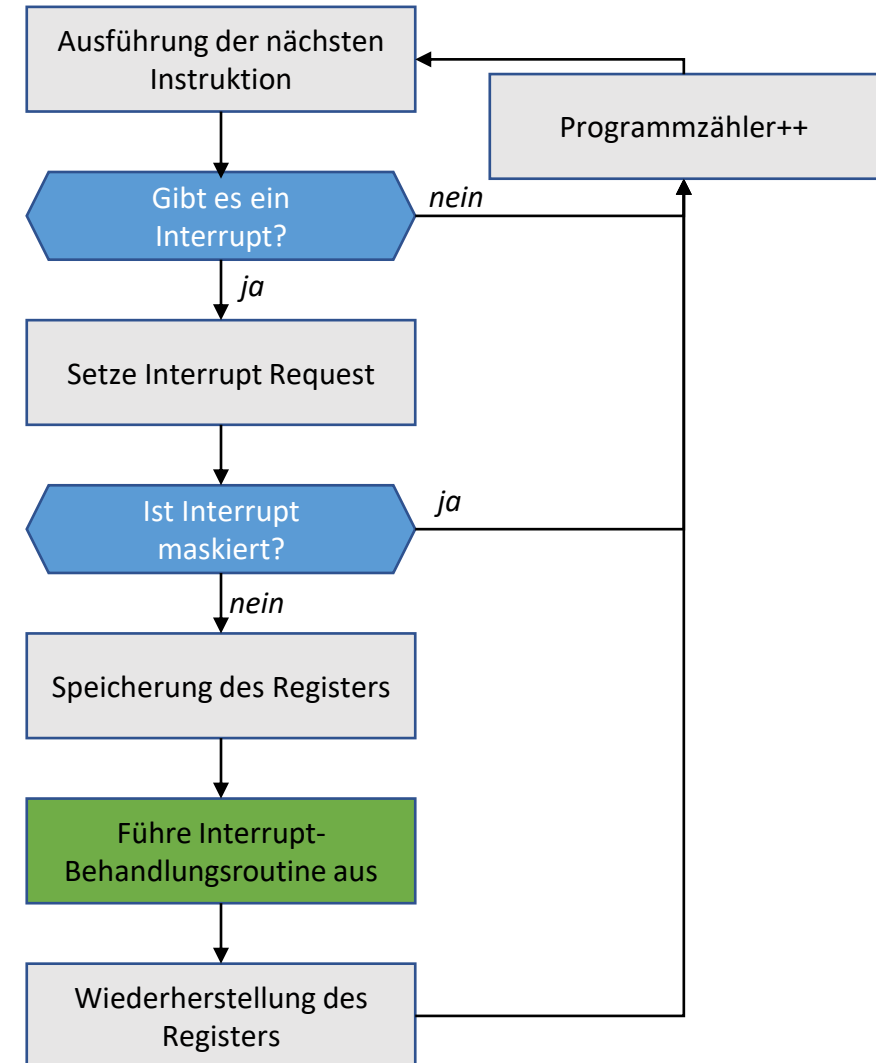


Unterbrechungen

- Unterbrechungen (*Interrupts*) stellen die Möglichkeit dar, einen Prozess zu unterbrechen, um eine andere, meist kurze, aber zeitkritische Verarbeitung durchzuführen
- Auslösendes Ereignis nennt man Unterbrechungsanforderung (Interrupt Request oder IRQ)
- Zwei Arten der Unterbrechungsanforderung
 - Möglichkeit für I/O-Werk, Prozessor über Zustand eines I/O-Gerätes zu informieren, z.B. über einen vollen Ausgabepuffer oder über ein neu empfangenes Datenpaket (externe Unterbrechung)
 - Möglichkeit für Prozessor, Ausnahmebehandlungen durchzuführen, z.B. bei Division durch 0 (interne Unterbrechung)
- Programmierbare Timer können nach Ablauf einer vorgebbaren Zeitspanne eine Unterbrechung anfordern (Scheduling)

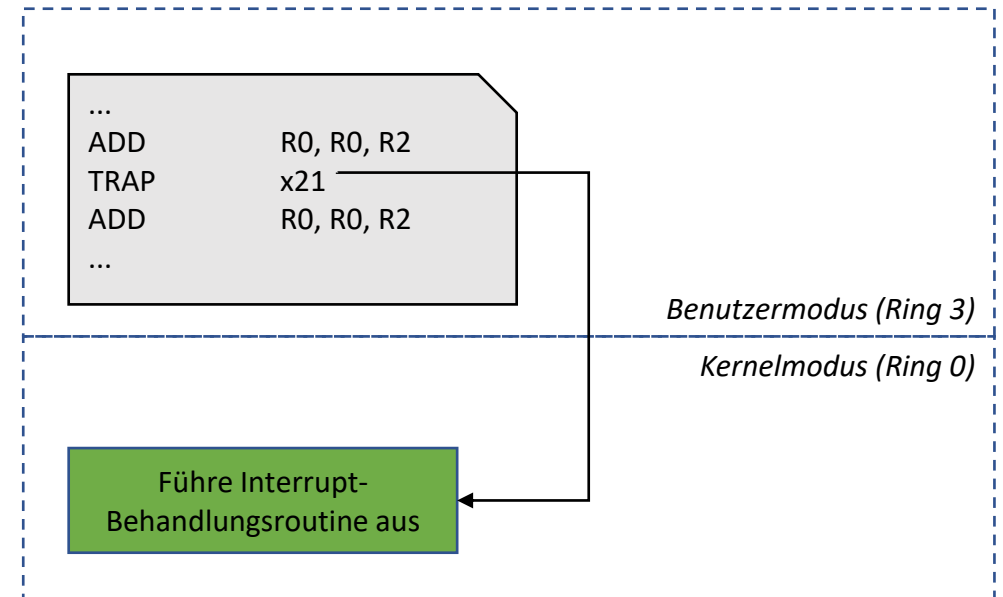
Allgemeiner Ablauf einer Unterbrechung

- Unterbrechung ist nur nach Ausführung einer Maschinen-Instruktion möglich
- Moderne Prozessoren verfügen über Instruktionen, um Unterbrechungen zu verbieten (**maskieren**) bzw. zu erlauben
- Interrupt-Behandlungsroutine (Interrupt Service Routine oder ISR) wird im Kernelmodus ausgeführt



Systemaufrufe und Unterbrechungen

- Interne Unterbrechung kann auch der Implementierung von Systemaufrufen („Traps“) dienen
- Instruktionen einer CPU mit Kernel und Benutzermodus können gruppiert werden in
 - Kontrollsensitive Instruktionen: Änderung der Ressourcenkonfiguration des Systems
 - Verhaltensensitive Instruktionen: Ausgabe oder Verhalten abhängig von der Ressourcenkonfiguration
 - Privilegierte Instruktionen: Verursachen ein Trap/Interrupt bei Ausführung in Benutzermodus



Architekturen

Kompakte Wiederholung der bisherigen Kapitel + Ausblick Architekturen

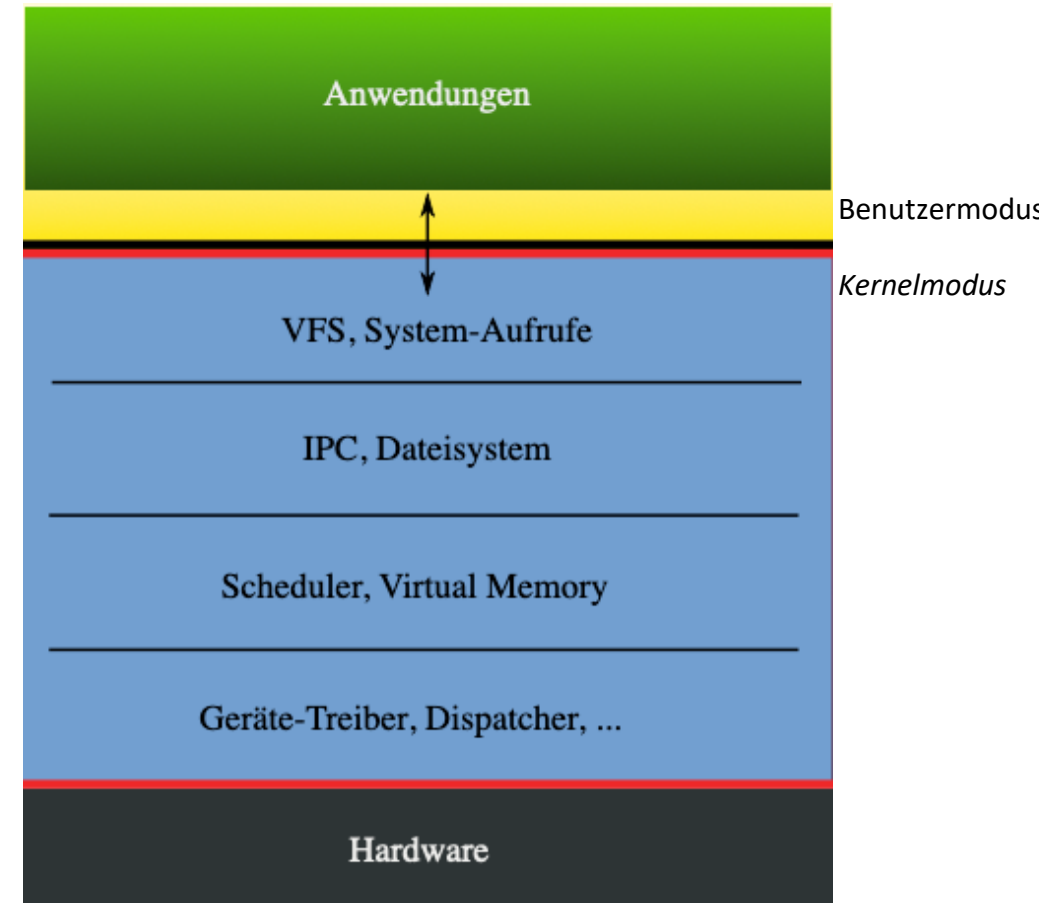
<https://www.youtube.com/watch?v=nfDEPxNNQ2M&t=201s>

Betriebssystem Architekturen

- Die **Architektur** eines Betriebssystems legt fest, wie die internen Module zueinander organisiert sind
- Typische Architekturen
 - Monolithische Architektur
 - Mikrokern Architektur
 - Hybride Architektur

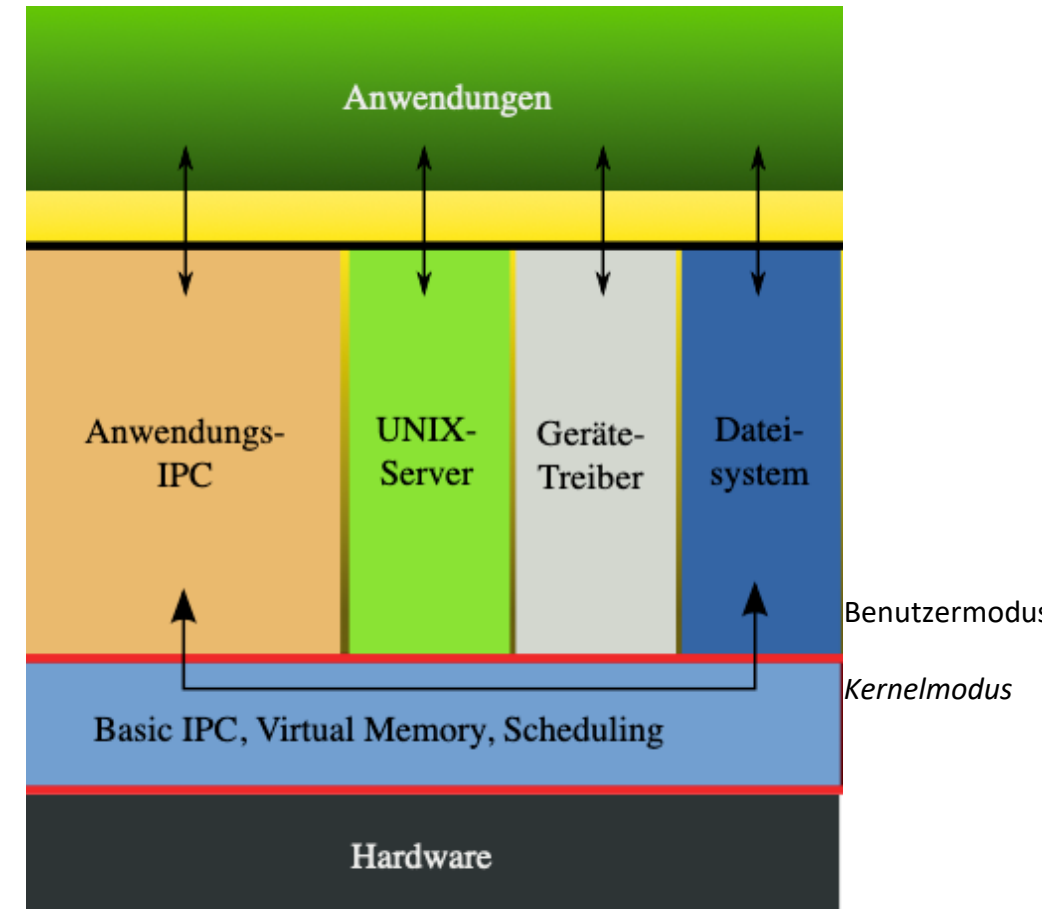
Monolithische Architektur

- Alle Module werden im Kernelmodus ausgeführt
- Ein Dienstverteiler nimmt Aufrufe entgegen und verteilt sie an zuständige Module
- Viele Betriebssysteme unterstützen dynamisches Laden von Modulen nach Bedarf
 - Unix: Shared libraries
 - Windows in `C:\Windows\system32`: Dynamic-Link Libraries (.dll)
- Beispiele: MS-DOS, Windows 9x, Linux, BSD-Unix, ...



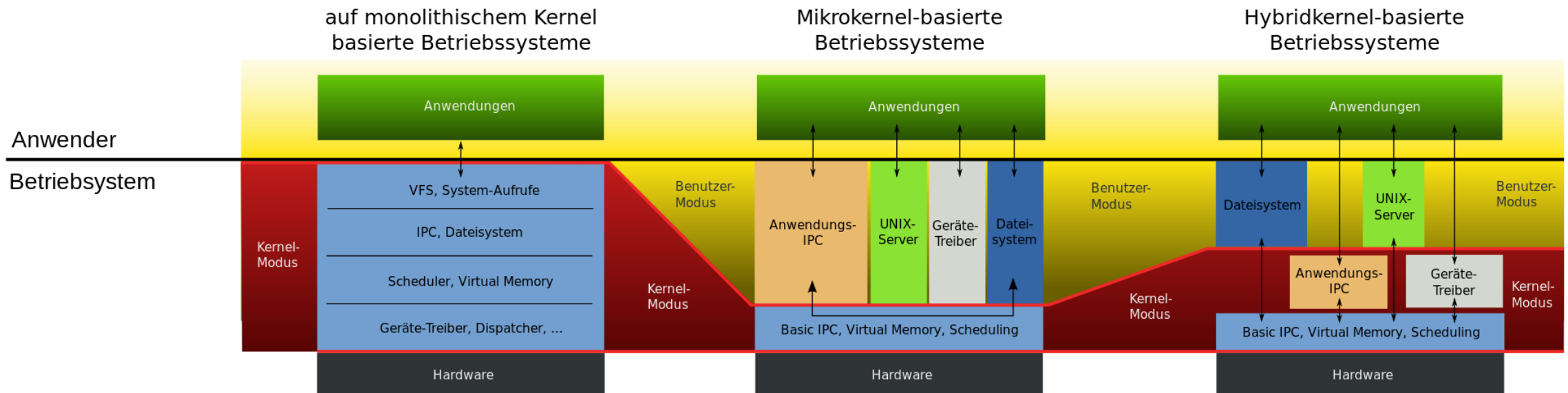
Mikrokern Architektur

- Mikrokern Architektur hat einen schlanken Kern, der im Kernelmodus läuft
- Funktionalität des Kernels (z.B. Datei- oder Speicherverwaltung) wird in Serverprozesse ausgelagert, die im Benutzermodus laufen
- Mikrokern übernimmt grundlegende Funktion (wie z.B. Prozessverwaltung) und die Kommunikation zwischen Anwendung (Client) und Serverprozessen
- Vergleich zu monolithischen Kernel
 - weniger Lines of Code (LoC)
 - Langsamer (mehr Kontextwechsel)
- Beispiele: Symbian OS, QNX



Hybride Architektur

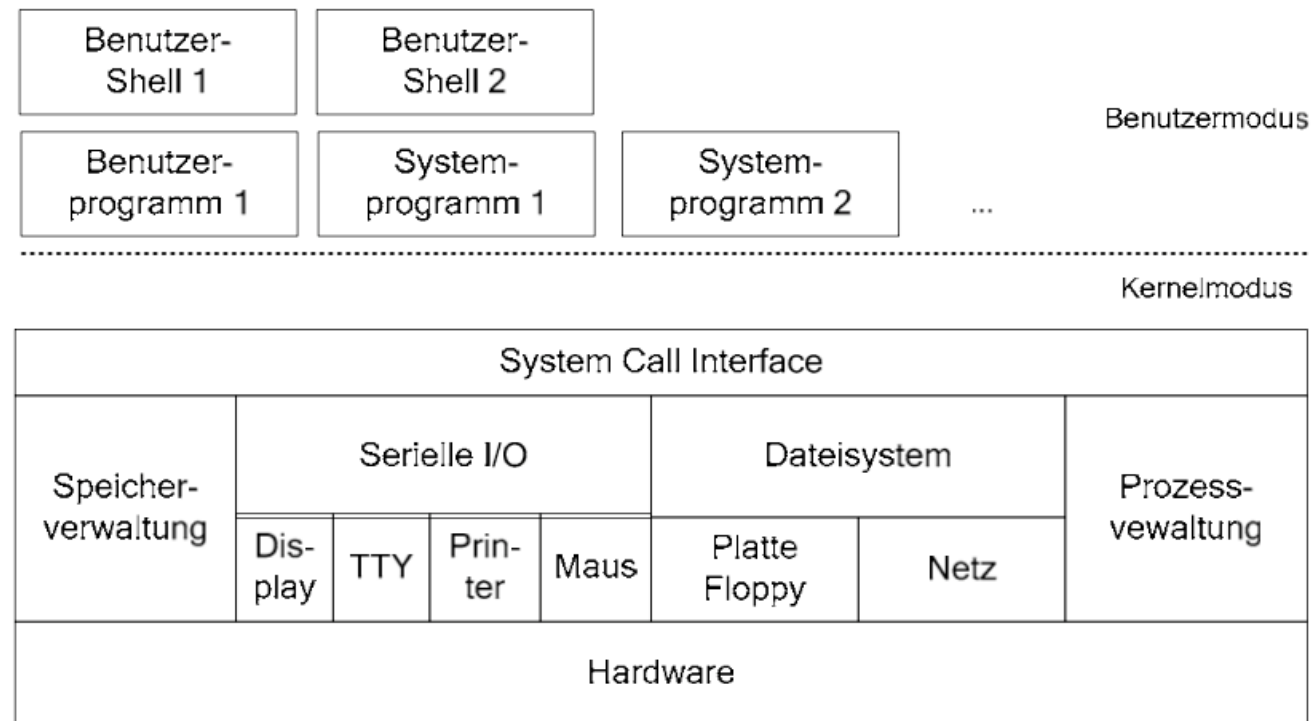
- Bei der Architektur mit Hybrid-Kern laufen im Gegensatz zum Mikrokern alle wichtigen Komponenten im Kernelmodus



- Beispiele: Windows NT (Windows XP, Windows 7, Windows 10, Windows 11, ...)

Unix Architektur

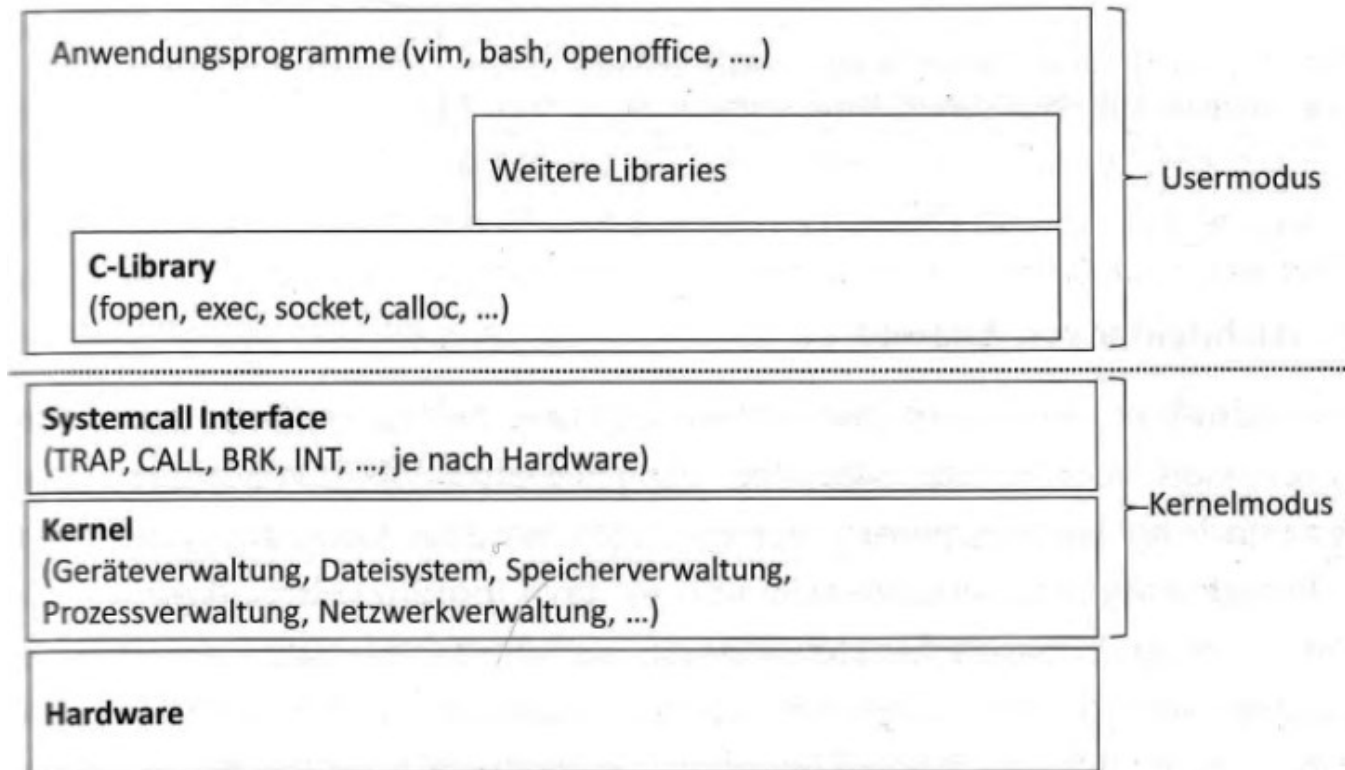
- Benutzerprogramme werden i.d.R. von einer Shell gestartet
- Dateien und I/O Geräte werden logisch einheitlich behandelt



Quelle: P. Mandl – Grundkurs Betriebssysteme

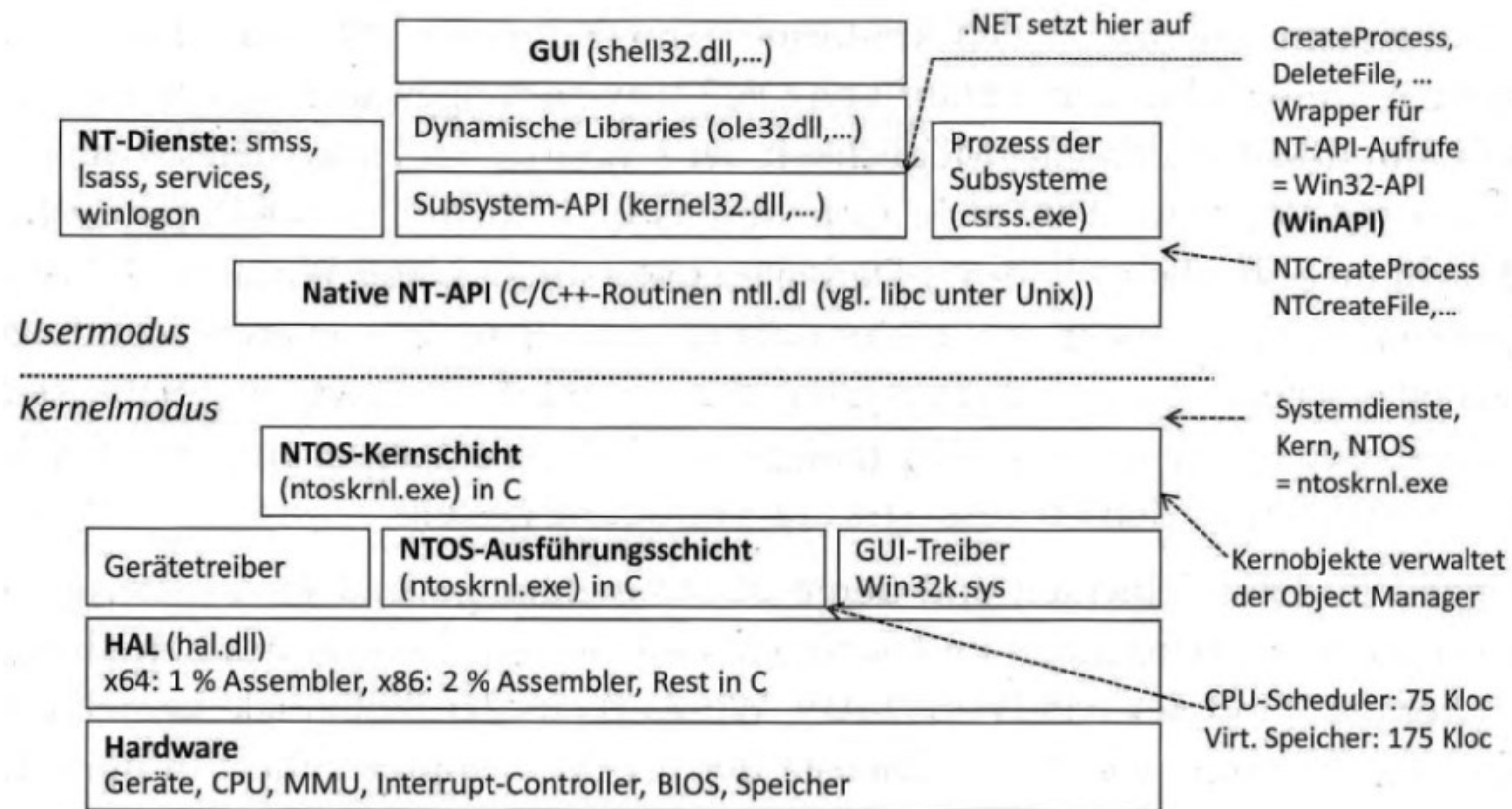
Linux Architektur

- Ähnlich zu Unix Architektur



Windows NT Architektur

- Vielschichtige, unübersichtliche Architektur mit Hybrid-Kern



Windows NT Architektur

- Hardwareabstraktion durch zusätzliche HAL-Schicht
- Anwendungen nutzen Windows-API aus der Bibliothek *kernel32.dll*, die Dienste aus der NT-API in *ntdll.dll* aufruft
- Es gibt insgesamt über 800 DLLs, die mehr als 13000 Systemfunktionen beinhalten und über 130 MByte groß sind
- Windows-API Aufrufe erfolgen teilweise in Benutzermodus
- Eigentlicher Kern in *ntoskrnl.exe*, stellt mehr als 1200 Systemaufrufe zur Verfügung
- Hoher Aufwand für Kompatibilität zu früheren Windows-Versionen (bis auf Windows 95 zurück)
- Windows XP, 7, 8 und 10 basieren auf Windows NT
- Heute insgesamt ca. 50 Mio. Codezeilen (Linux hat etwa 2 Mio. Codezeilen)

Quiz



"Dieses Foto" von Unbekannter Autor ist lizenziert gemäß [CC BY-SA-NC](#)